

Laboratory 2 – Diving deeper with Neo4j

1. Local Infrastructure Setup and Troubleshooting

The first step of the project involved configuring the local development environment using Docker Compose. We deployed two containers: a Neo4j database limited to 3GB of RAM and a Java application. During this phase, we resolved container initialization issues by removing an empty local `/conf` volume mount that was overriding Neo4j's default configuration and causing it to crash.

2. JSON Data Parsing Strategy

To process the DBLP dataset without exceeding the strict memory limits, we utilized the `jackson-databind` library. Because the data is provided in JSON Lines format (`.jsonl`), our Java application reads the file line-by-line using a `BufferedReader`. Which makes that only one article is loaded into memory at a time. Each line is parsed into a `JsonNode` tree, allowing us to safely extract the `key_id`, `title`, `authors`, and `references`.

3. Database Constraints and Indexing

Before inserting any data, we configured the database schema by establishing unique constraints. Using Cypher, we created constraints enforcing that every `ARTICLE` node has a unique `key_id` and every `AUTHOR` node has a unique `name`. This step reduces the lookup time during node insertion.

4. Cypher Query Construction (Using MERGE)

To construct the graph correctly, we implemented Neo4j `MERGE` queries instead of `CREATE` statements. This handles two major characteristics of the dataset:

- **Multiple Authorships:** Authors frequently write multiple papers. `MERGE` ensures that an `AUTHOR` node is only created once, and subsequent papers simply draw an `AUTHORED` relationship to the existing node.
- **Forward Citations:** Articles often cite papers that appear later in the dataset. `MERGE` allows to create placeholder `ARTICLE` nodes using only the `key_id`. When the article is parsed, it updates the placeholder with the right title using the `ON MATCH SET`.

5. Local Testing and Verification

We tested locally with a small subset (the first 5 records). The Java application created `ARTICLE` and `AUTHOR` nodes, and linked them via `AUTHORED` and `CITES` relationships. We verified the resulting graph structure using the Neo4j Browser, confirming that the nodes and

relationships matched the expected data model. Like shown in Fig 1.

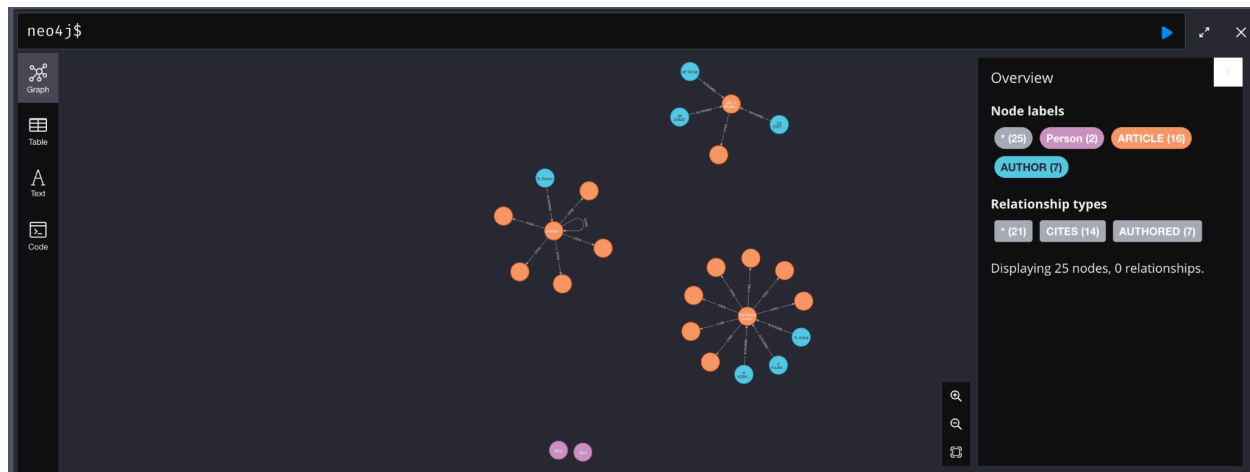


Figure 1. Return of a sample of 25 nodes along with their interconnected relationships.

Logic of the results:

- **ARTICLE Nodes (Orange):** The query successfully retrieved 16 **ARTICLE** nodes. Although we only processed 5 lines of JSON data, 16 nodes exist because our Cypher **MERGE** logic creates placeholder nodes for references, initializing them with their **key_id** even before the full article data is parsed.
- **AUTHOR Nodes (Blue):** The graph contains 7 **AUTHOR** nodes extracted from the parsed dataset.
- **AUTHORED Relationships:** The visualization confirms the creation of 7 **AUTHORED** relationships, which correctly connect the **AUTHOR** nodes to the **ARTICLE** nodes.
- **CITES Relationships:** The graph shows 14 **CITES** relationships, linking **ARTICLE** nodes to the other **ARTICLE** nodes they reference.
- **Person Nodes (Purple):** The 2 **Person** nodes visible in the graph are remnants of the dummy node created by the initial boilerplate code provided in the example repository.

6. Batch Processing

To handle the 18GB dataset and adhere to the strict 3GB memory limit of the Neo4j container, we upgraded the ingestion logic from single-record processing to **Batch Processing**.

Sending individual database transactions for millions of records introduces severe network latency and overhead. To solve this, we implemented the following strategy:

- **In-Memory Chunking:** The Java application reads the JSON Lines file sequentially and maps the extracted variables (**id**, **title**, **authors**, and **references**) into a Java **HashMap**. These maps are accumulated into a **List** until a predefined batch size (e.g., 5,000) is reached.

- **The UNWIND Clause:** Instead of running the 5,000 separate queries, the entire list is passed to Neo4j as a single parameter. The Cypher **UNWIND** command is then used to unpack this list directly on the database server, processing the entire batch in one transaction.

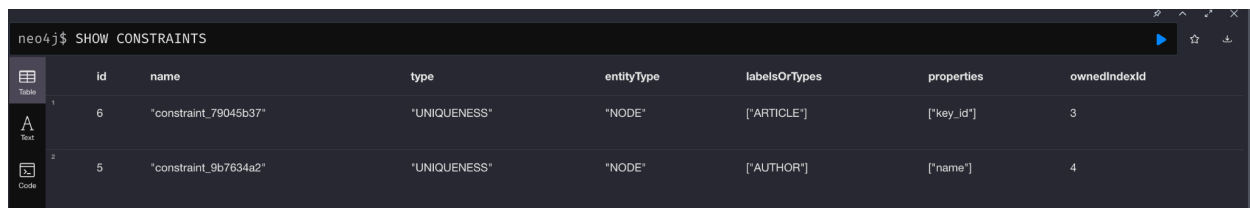
7. Performance Monitoring and Logging

we implemented logging metrics within the Java application.

- The application captures system timestamps (`System.currentTimeMillis()`) when the parsing begins and ends, calculating the total duration of the data loading process.
- Once the final batch is committed, the application opens a read transaction to query the database for the total number of ingested nodes (`MATCH (a:ARTICLE) RETURN count(a)` and `MATCH (auth:AUTHOR) RETURN count(auth)`). This provides a reliable summary of the total authors, articles, and overall graph size in the console logs.

8. Verification of Database Constraints

Before reading the dataset, the application executes commands to enforce uniqueness constraints. We verified this using the **SHOW CONSTRAINTS** query in the Neo4j Browser.



	id	name	type	entityType	labelsOrTypes	properties	ownedIndexId
1	6	"constraint_79045b37"	"UNIQUENESS"	"NODE"	["ARTICLE"]	["key_id"]	3
2	5	"constraint_9b7834a2"	"UNIQUENESS"	"NODE"	["AUTHOR"]	["name"]	4

Figure 2. Return of the 2 constrains.

The verification confirmed the active status of two primary constraints:

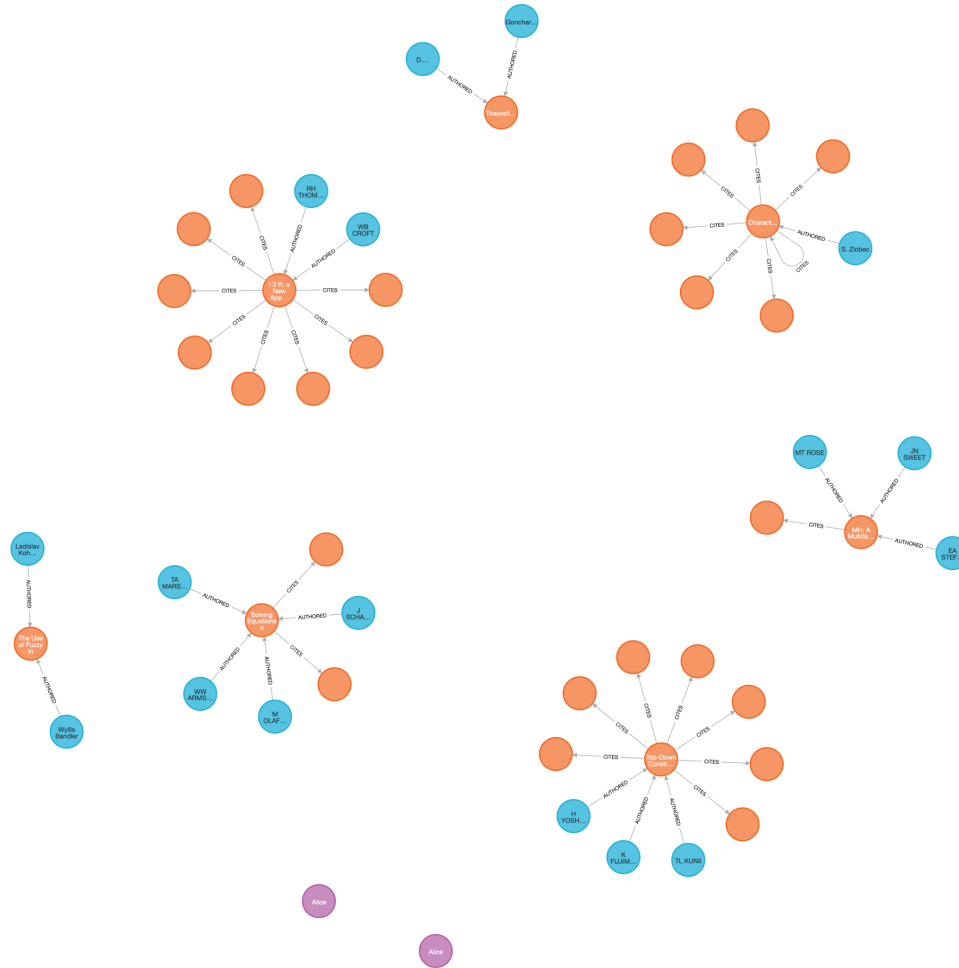
1. **ARTICLE Constraint (key_id):** Ensures that no two articles can share the same unique ID. If an article is referenced by multiple papers, the database merges the relationships onto a single node rather than duplicating it.
2. **AUTHOR Constraint (name):** Ensures that author nodes are unique. If a specific author publishes multiple papers, the database links the new articles to the pre-existing author node instead of creating duplicates.

The Implicit Indexing Advantage:

Indexing the `key_id` and `name` properties, Neo4j can locate existing nodes in milliseconds, allowing our batch ingestion process to maintain high speed regardless of the database size.

Testing:

```
MATCH (n) RETURN n LIMIT 50
```

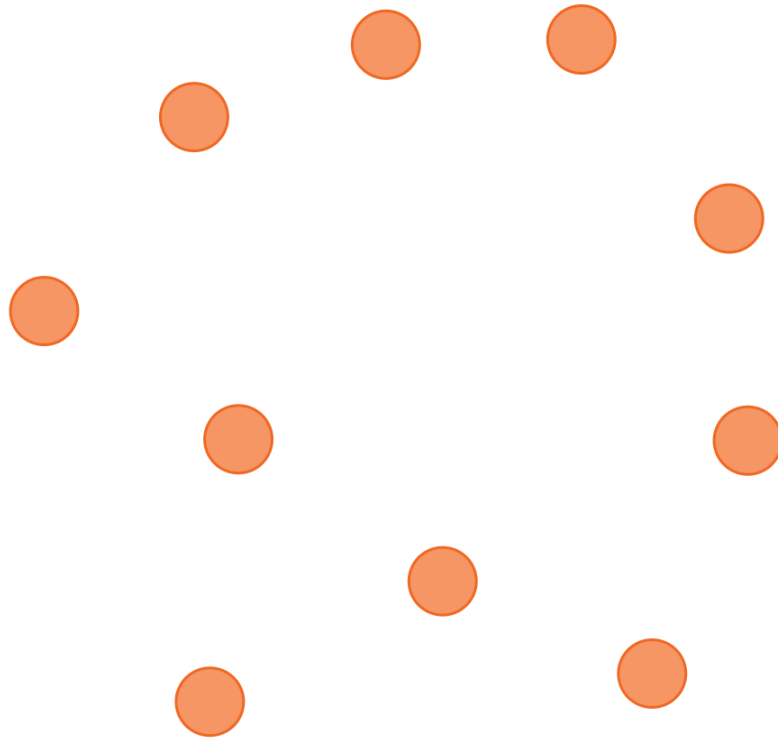


Explanation: These visualizations display an interconnected network representing the core data model. The query `MATCH (n) RETURN n LIMIT 25` instructs Neo4j to grab a random sample of up to 25 nodes from the database and display them, along with any relationships that exist *between* those specific nodes.

Looking at the graphs, we can identify that:

- **Blue Nodes (AUTHOR):** These represent the individuals who wrote the papers.
- **Orange Nodes (ARTICLE):** These represent the scientific publications.
- **AUTHORED Relationships (Arrows from Blue to Orange):** These directional arrows show exactly which author wrote which article. We can see instances of multiple authors pointing to a single central article, demonstrating co-authorship.
- **CITES Relationships (Arrows from Orange to Orange):** These arrows show the citation network, where one article references another. In the figure above, we can see central articles surrounded by a cluster of other articles they cite.

```
MATCH (a:ARTICLE) WHERE a.title IS NULL RETURN a LIMIT 10
```



Node properties 

ARTICLE

<id>	8	
key_id	5390a1e620f70186a0e59c05	

Explanation: This image displays a cluster of isolated orange **ARTICLE** nodes with no connections to authors or other papers. This visualization shows the concept of the "**Placeholder Nodes**" created by the Cypher **MERGE** logic.

When our batch processing script reads an article, it extracts the `references` (citations) array. Because an article might cite a paper that appears much later in the 18GB dataset, our script uses `MERGE (cited:ARTICLE {key_id: refId})` to create that cited article immediately, using only the ID.

Because the full data for that cited paper hasn't been read yet, it does not yet have a title, nor does it have its own authors attached to it. It exists purely as a structural anchor in the database. When the script eventually reaches that specific paper's data later in the file, the `ON MATCH SET a.title = row.title` command will populate the missing information and attach the respective authors, turning these isolated nodes into the fully connected clusters seen in the other images.

9. KUBERNETES

Namespace: rodri-adv-daba-26

Step: Streaming Data Directly from the URL

Instead of reading from a local `File`, the application utilizes `java.net.URL` and an `InputStreamReader`. This stream is passed into a `BufferedReader`, allowing the application to process the JSON lines one by one in memory. This ensures the application remains memory-efficient and avoids local disk storage overload.

Step: Containerization and Cross-Platform Build

The Java application was packaged into a Docker container. Because the local development environment used an Apple Silicon (ARM64) architecture, the image was specifically compiled for the target Kubernetes cluster using the `--platform linux/amd64` flag. The compiled image was then pushed to a public Docker Hub repository to allow the remote Kubernetes nodes to easily pull the image without requiring secret management. This due to Apple environment efficiency.

Step: Kubernetes Deployment in dashboardsd

The translation from docker yaml to kubernetes cluster was made through Rancher the dashboard provided and divided in 2 configurations:

1. **Database Deployment:** A Kubernetes `Deployment` and `Service` were created for Neo4j. As per the strict laboratory constraints, resource limits were in the YAML (`memory: "3Gi"` and `cpu: "2"`).
2. **Data Loader Job:** A Kubernetes `Job` was used to execute the Java container. Environment variables were used to pass the database URL (`neo4j-service`), the dataset URL, and the maximum node limit.

Test with a max node of: 10000

```
dblp-loader-job-ttx9 x neo4j-deployment-679bffd49d-hhcjb x
Mon, May 4 2026 10:45:27 pm WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
Mon, May 4 2026 10:45:27 pm Sleeping a bit waiting for the db
Mon, May 4 2026 10:45:32 pm Creating database constraints/indexes...
Mon, May 4 2026 10:45:34 pm Constraints created successfully.
Mon, May 4 2026 10:45:34 pm Streaming data directly from URL: http://vmrum.isc.heia-fr.ch/files/DBLP-Citation-network-V18.jsonl
Mon, May 4 2026 10:45:34 pm Reading the json file and writing to Neo4j in batches...
Mon, May 4 2026 10:45:34 pm --- LOADING STARTED AT: 1777927534843 ms ---
Mon, May 4 2026 10:45:46 pm Inserted batch of 5000 records. Total processed: 5000
Mon, May 4 2026 10:45:54 pm Inserted batch of 5000 records. Total processed: 10000
Mon, May 4 2026 10:45:54 pm --- LOADING ENDED AT: 1777927554079 ms ---
Mon, May 4 2026 10:45:54 pm Total loading time: 19 seconds
Mon, May 4 2026 10:45:54 pm Total ARTICLES loaded: 95470
Mon, May 4 2026 10:45:54 pm Total AUTHORS loaded: 30732
Mon, May 4 2026 10:45:54 pm Total nodes loaded (Articles + Authors): 126202
Mon, May 4 2026 10:45:55 pm [INFO] -----
Mon, May 4 2026 10:45:55 pm [INFO] BUILD SUCCESS
Mon, May 4 2026 10:45:55 pm [INFO] -----
Mon, May 4 2026 10:45:55 pm [INFO] Total time: 30.128 s
Mon, May 4 2026 10:45:55 pm [INFO] Finished at: 2026-05-04T20:45:55Z
Mon, May 4 2026 10:45:55 pm [INFO] -----
```

State	Name	Namespace	Image	Ready	Restarts	IP	Node	Age
Error	dblp-loader-job-pqjx5	rodri-adv-daba-26	diosarodriguez22/neo4jp	0/1	0	10.42.21.58	k8s-node5	17 mins
Completed	dblp-loader-job-ttx9	rodri-adv-daba-26	diosarodriguez22/neo4jp	0/1	0	10.42.11.213	k8s-node12	9 mins
Running	neo4j-deployment-679bffd49d-hhcjb	rodri-adv-daba-26	neo4j:4.4.15-community	1/1	0	10.42.17.134	k8s-node6	26 mins

Git: https://github.com/Diosa-Rodriguez/Neo4j_Labo_AdvDB.git